

PROJECT REPORT

ON

“USER – ROLE AUTHENTICATION”

SUBMITTED IN THE PARTIAL FULFILLMENT FOR THE AWARD OF
DEGREE OF BACHELOR OF COMPUTER APPLICATION



UTTARANCHAL
UNIVERSITY
DEHRADUN

NAAC
GRADE **A+**

UTTARANCHAL SCHOOL OF COMPUTING SCIENCES

UTTARANCHAL UNIVERSITY

Session (2025-2026)

Under the supervision of:

Mr. Deepak Kumar

(Assistant Professor)

USCS

Submitted by:

Suhani Chaudhary

Bca (2nd semester)

Section : - H

ACKNOWLEDGEMENT

I express my thanks to **Mr. Deepak Kumar, Project Mentor, and Assistant Professor-USCS**. He has been generous enough to provide me an opportunity and accepting my candidature for the most valuable guidance and affordable treatment given to us at every stage to boost my morale.

The most awaited moment of any Endeavour is successful completion, but nothing can be done successfully if done alone. Success is the outcome of contribution and consistent help of various persons and we thank those ones who helped us in successful completion of this project.

Primarily I would like to thank **Prof (Dr.) Sonal Sharma, Director-USCS** for providing a healthy and encouraging environment to study.

I profusely thankful to the **Prof (Dr.) Sameer Dev Sharma, HoD-USCS** for providing all the required resources in the successful completion of my project

Suhani Chaudhary
B.C.A(2nd SEMESTER)
Enroll No. UU2509000561

DECLARATION

I hereby declare that the project report titled “**User-Role Authentication**” was submitted Suhani Chaudhary to Uttarakhand School of Computing Sciences. The project was done under the guidance of **Mr. Deepak Kumar**. I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other university or institute

Suhani Chaudhary
B.C.A(2nd SEMESTER)
Enroll No. UU2509000561

CERTIFICATE OF ORIGINALITY

This is to certify that the project entitles “**User- Role Authentication**” by **Suhani Chaudhary** has been submitted in the partial fulfilment of the requirements for the award of the degree of BCA from department of computer applications from Uttarakhand school of computing science **Uttarakhand University**, Dehradun. The results embodied in this project have not been submitted to any other University or Institution for the record of any degree.

Under the guidance of:

Mr. Deepak Kumar

Assistance Professor-USCS

Uttarakhand University Dehradun

TABLE OF CONTENT

1. ACKNOWLEDGEMENT	2
2. DECLARATION	3
3. CERTIFICATE OF ORIGINALITY	4
4. INTRODUCTION	6
5. OBJECTIVE	7
6. PURPOSE	8
7. IDENTIFICATION OF NEED	10
8. TOOLS AND PLATFORMS	11
9. PROBLEM STATEMENTS	12
10. SYSTEM ANALYSIS	13
11. FEASIBILITY STUDY	14
12. PROJECT PLANNING	15
13. PROJECT DESIGN	16-17
14. PRILIMINARY INVESTIGATION	18
15. FLOWCHART	19
16. GANT CHART	20
17. DFD DIAGRAM	21
18. CODE	22-34
19. TESTING	35
20. CONCLUSION	36
21. BIBLOGRAPHY	37

INTRODUCTION

User Role Authentication is a system that controls access to resources based on the roles assigned to individual users. user role authentication typically involves the identification of a user and the verification of their access privileges based on pre-defined roles (e.g., Admin, User, Guest). The system ensures that users can only access certain features or data that are permitted according to their roles.

we implement a basic user authentication system where users are prompted to enter their credentials (such as username and password). Upon successful login, the system assigns them a role, and based on their role, grants or restricts access to specific functionalities within the application.

The project involves:

1. **User Registration:** Users can register with a username, password, and role.
2. **Login:** Users authenticate themselves by entering their credentials.
3. **Role-based Access Control:** Depending on the user's role (Admin, User, or Guest), they will have different levels of access to system features or resources.

By incorporating role-based access control, this mini project helps in understanding the importance of user roles and access security, providing a solid foundation for more complex systems requiring authentication and authorization.

This simple system can be extended to include password encryption, session management, and persistent storage (such as file or database), depending

OBJECTIVE

The objective of this project is to develop a **User Role Authentication System** in C++ that ensures secure access control based on user roles. The system will authenticate users and authorize their access to different functionalities depending on their assigned roles, such as **Administrator, Standard User, and Guest**.

The key objectives of this project include:

1. **User Authentication:** Implement a login system where users must enter valid credentials to access the system.
2. **Role-Based Access Control (RBAC):** Assign different permissions to users based on their roles, restricting unauthorized access to sensitive functions.
3. **Data Security:** Ensure secure storage and retrieval of user credentials, using encryption or hashing techniques.
4. **System Scalability:** Design the system to allow easy modification of roles and permissions as needed.
5. **User-Friendly Interface:** Develop a clear and efficient interface for users to interact with the system, whether console-based or GUI-based.

This project aims to demonstrate the importance of role-based authentication in software security by providing a structured and efficient mechanism for managing user access.

PURPOSE

The primary purpose of this project is to **authenticate users** and **authorize access** to different system functionalities based on their assigned roles, such as **Administrator, Standard User, or Guest**.

By implementing this system, the project aims to:

- **Prevent unauthorized access** by ensuring only authenticated users can log in.
- **Restrict functionalities** based on user roles to enhance security and maintain system integrity.
- **Demonstrate secure authentication methods** such as password validation and encryption.
- **Improve access management efficiency** by allowing easy modification of user roles and permissions.

This project serves as a practical demonstration of security principles in software development, ensuring data protection and controlled system access through structured authentication and authorization mechanisms.

FUTURE SCOPE

The future scope of the **user role authentication** project has a lot of exciting possibilities for improvement.

1. **cloud integration**, which would allow the system to scale smoothly as more users come on board and make it easier to manage access across different devices and platforms.
2. Adding **real-time monitoring and alerts** would make the system more responsive, sending admins an instant notification if something suspicious happens—like an unusual login attempt—so they can act quickly.
3. improving **encryption and privacy** measures would ensure that sensitive information, like passwords and biometric data, stays protected and private, meeting the highest security standards.

With all these features, the system would become more secure, user-friendly, and ready for real-world applications.

IDENTIFICATION OF NEED

Identifying the **need for user role authentication** is crucial for controlling access and ensuring proper functionality. Here's a breakdown of why it's important and how it applies to your project:

Why is User Role Authentication Needed?

- **Security Control**
 - Only authorized users (e.g., admins) should have the power to approve or block USB devices.
 - Prevents attackers or unauthorized users from overriding security mechanisms.
- **Data Protection**
 - Ensures that sensitive system settings or logs are only accessible to trusted users.
- **Accountability**
 - With proper role-based authentication, logs can track which user allowed or blocked a device.
- **System Integrity**
 - Prevents accidental or malicious changes by unauthorized users.

TOOLS AND PLATFORM

1. Development Tools

- **IDE/Editor:** Visual Studio Code, Code::Blocks, CLion, or Qt Creator (if using Qt)
- **Compiler:** GCC (MinGW for Windows)

2. Libraries & Frameworks

- **Standard Libraries:** iostream (file handling), vector, stdlib, string, SQLITE3
- **SQLite:** to store habits in local database
- **GUI Frameworks (Choose One):**
- **Qt:** Provides cross-platform UI with extensive widgets

3. Platforms

- **Windows:** Can use MinGW or MSVC

PROBLEMS STATEMENTS

1. Basic Problem Statement:

aims to develop a secure and efficient user role authentication system that verifies user identities, assigns roles, and enforces appropriate access controls."

2. Key feature:

User Login System

- Secure login mechanism with username and password validation.

Role-Based Access Control (RBAC)

- Define multiple roles such as Admin, Moderator, and User.
- Assign different permissions and access levels to each role.

User Privilege Verification

- Check if the logged-in user has admin rights, standard user access, or guest permissions.

SYSTEM ANALYSIS

The User Role Authentication System aims to provide secure access control by verifying user identities and assigning appropriate roles. This system ensures that only authorized users can perform specific actions, enhancing overall security and user management.

User Role Authentication System is to provide a secure and efficient method of verifying user identities and controlling access to system resources based on assigned roles.

Key Objectives

- Access Control: To restrict unauthorized users from accessing sensitive data or performing privileged actions.
- Security Enforcement: To implement robust authentication mechanisms that prevent unauthorized logins and privilege escalation.
- Role-Based Management: To assign predefined roles (e.g., Admin, User, Guest) with specific permissions, ensuring clear responsibility distribution.
- Data Protection: To securely encrypt user credentials and maintain confidentiality.
- Audit and Accountability: To track user activities for monitoring and investigation purposes.

FEASIBILITY STUDY

1. Technical Feasibility

Advantages:

- **High Performance & Efficiency:** C++ is a compiled language, making it faster
- **Cross-Platform Compatibility:** Can be used on Windows, Linux, and MacOS with minimal modifications.
- **Oriented Object Programming (OOP):** Supports modular programming, making it suitable for scalable applications.
- **Extensive Libraries & Frameworks:** Supports GUI (Qt, wxWidgets),

2. Financial Feasibility

- **Cost Benefits:**
- **Development Tools:** Open-source Compilers (GCC, Clang) — Open-source IDEs (Visual Studio, Code::Blocks, Qt Creator) somewhat reduce development cost
- **No Licensing Fees:** C++ is open-source, meaning it is free to use compared to proprietary software.
- **Long Live To Tell The Tale:** C++ applications are generally designed to run for an extended period, with little to change.

3. Operational Feasibility

- **Usability**
 - **Admins** can manage roles, configure permissions, and access sensitive data.
 - **Users** can securely log in and access only authorized features.
 - **Guests** are restricted to minimal or no access, preventing unauthorized actions.
- **Performance**
 - It also guarantees high performance – Large dataset can be operated smoothly on C++.

PROJECT PLANNING

Project planning is the process of outlining the steps, tasks, resources, and timelines necessary to complete a project successfully. In user role authentication project, planning is crucial for organizing the workflow, managing resources, and ensuring the project stays on track.

1. Defining Project Scope

- Clearly identifying the goals, objectives, and deliverables of the project.

2. Setting Milestones

- Breaking the project into smaller, manageable phases or tasks.

3. Creating a Timeline

- Defining the time frame for each milestone and task.

4. Budgeting and Cost Management

- Estimating the total cost of the project and managing the budget throughout development.
 - Development Costs: Include salaries for team members, costs for software, and any hardware required for biometric scanning.
 - Maintenance Costs: Factor in the long-term costs of maintaining the system (e.g., cloud hosting fees, system updates).
 - Unexpected Costs: Always include a buffer for unplanned expenses.

5. Testing and Quality Assurance

- Testing the system throughout the development process to ensure it works as expected and meets the requirements.

PROJECT DESIGN

Project design is a critical phase in the development of a user role authentication system. It involves creating a blueprint for how the project will be structured, how components will interact, and how the system will meet the defined requirements.

1. System Architecture Design

- The overall structure of the system, including how different components (such as the user interface, database, and authentication modules) interact with each other.
- User Interface (UI): Design a simple, intuitive interface for user registration, login, and role management. This can include:
- Authentication Flow:
 1. User enters login credentials or biometric data.
 2. The system verifies the data with stored information (using a database).
 3. If the credentials are valid, the user is granted access based on their assigned role.
 4. Role-based permissions control access to different parts of the system (admin, regular user, etc.).

2. Database Design

- The structure of the database that will store user information, credentials, and role details.
- Entities to be stored:
 - Users: Information like user ID, username, password (encrypted), and role.
 - Roles: Different roles (admin, user, etc.) and the permissions associated with each role.
- Database tables:
 - Users Table:
 - Columns: User_ID, Username, Password (hashed), Role_ID.
 - Roles Table:
 - Columns: Role_ID, Role_Name, Permissions.

4. User Roles and Permissions Design

- Defining the various user roles and their associated permissions within the system.

➤ Roles :

- Admin: Full access to the system, can assign roles, manage users, and configure settings.
 - Regular User: Can access basic features, but cannot change system settings or manage other users.
 - Guest (Optional): Limited access, perhaps only to view certain content.
- Permissions: Each role has specific actions they can perform, such as:
 - Admin: Create/assign roles, edit user info, view system logs.
 - Regular User: View personal profile, update password, access assigned content.
 - Guest: Read-only access to specific pages.

PRILIMINARY INVESTIGATION

The preliminary investigation of **user role authentication** in c++ involves project development where the idea is explored, feasibility is checked, and requirements are roughly identified.

1. Problem Identification

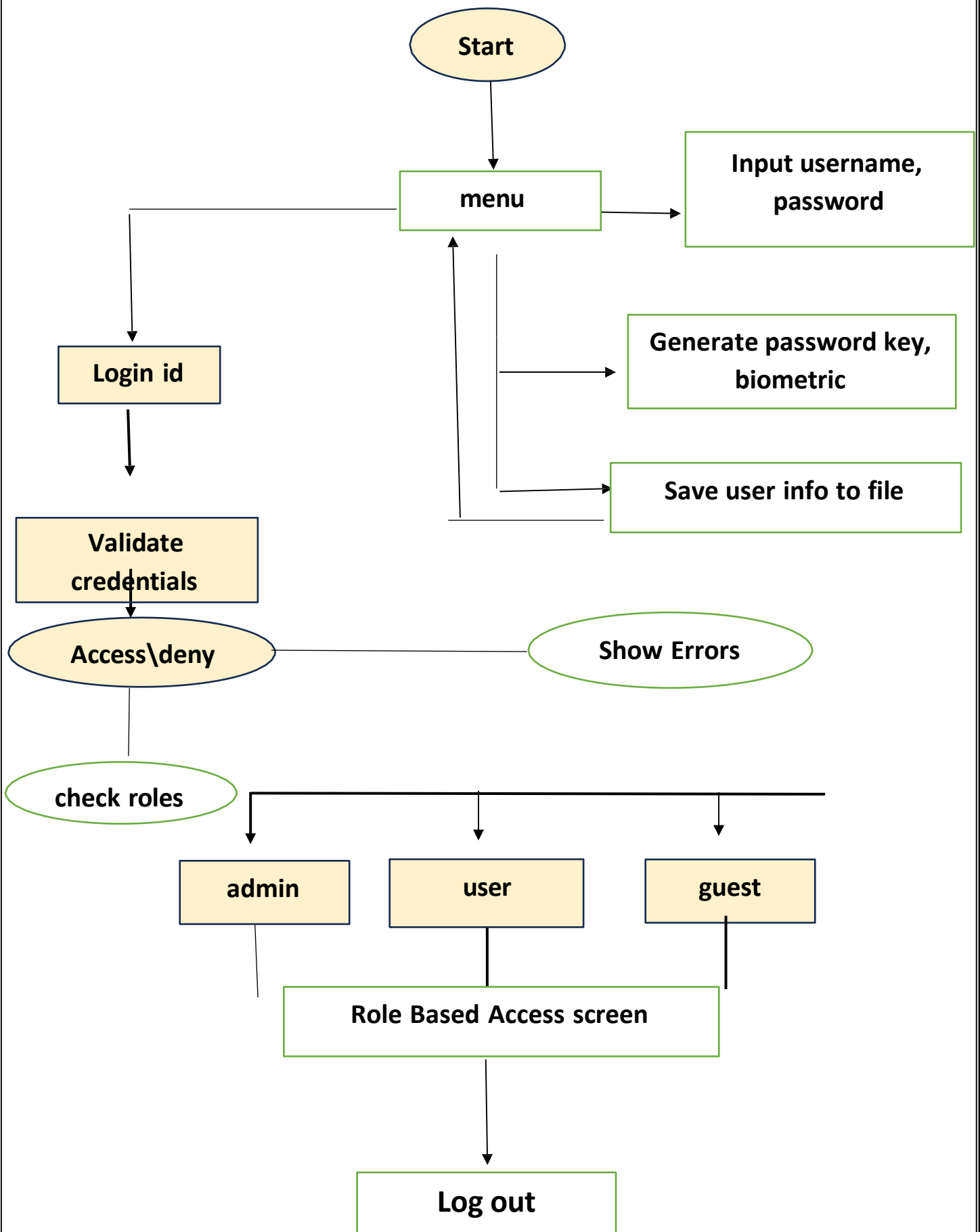
In **user role authentication** there was a need for a simple and secure user login system that could:

- Handle multiple users
- Offer role-based access
- Improve security beyond just a username and password

2. Requirement Identification

- User login system
- Role assignment
- Verification steps: Password → Passkey → Biometric (simulated)
- Output messages and access control

FLOWCHART



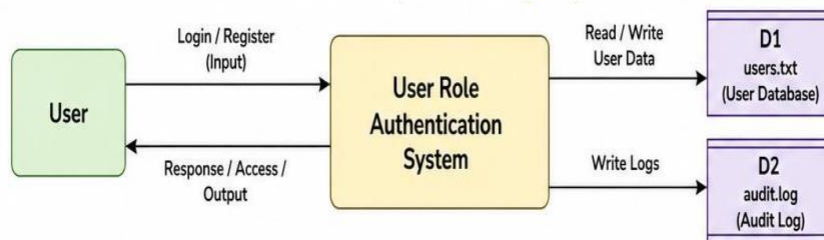
Gant chart

Task	Duration	Days
-----	-----	-----
Topic Selection	2 days	Day 1–2
Requirement Analysis	3 days	Day 3–5
System Design	2 days	Day 6–7
Database Design	3 days	Day 8–10
Frontend Development	5 days	Day 11–15
Backend Development	6 days	Day 11–16
Authentication Module	4 days	Day 15–18
Testing & Debugging	3 days	Day 19–21
Documentation	3 days	Day 22–24
Final Submission	1 day	Day 25

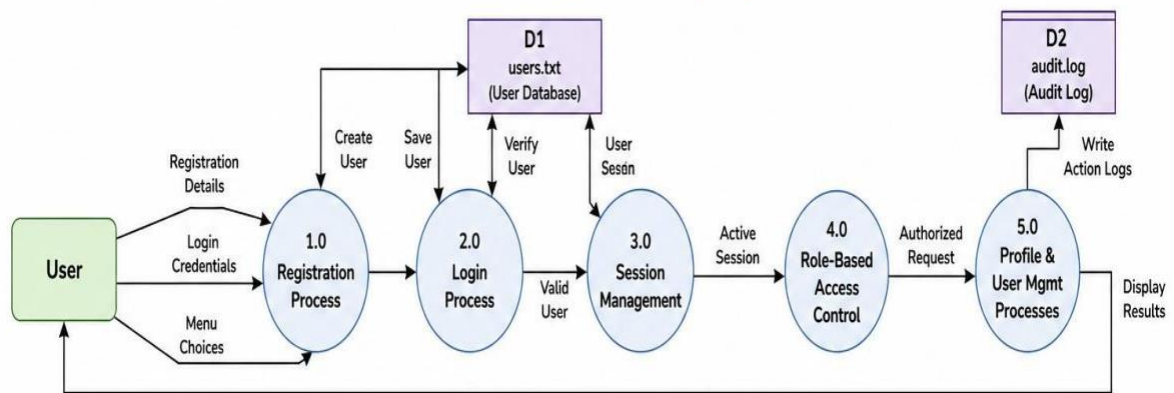
DFD

(Data Flow Diagram)

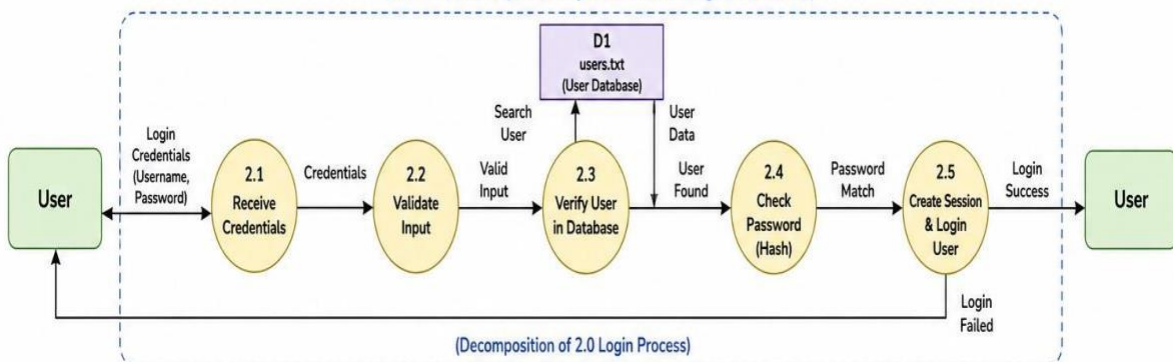
DFD Level 0 (Context Diagram)



DFD Level 1 (Detailed Diagram)



DFD Level 2 (Decomposition of Login Process)



CODING

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>
#include <chrono>
#include <iomanip>

using namespace std;

struct User {
    string username;
    string password;
    string role;
};

struct Session {
    bool active = false;
    chrono::system_clock::time_point lastActivity;
    User* user = nullptr;
};

string hashPassword(const string& password) {
    unsigned long hash = 5381;
    for (char c : password) {
        hash = ((hash << 5) + hash) + c;
    }
    stringstream ss;
```

```

    ss << hex << hash;
    return ss.str();
}

void logAction(const string& action, const string& username = "Unknown") {
    ofstream file("audit.log", ios::app);
    if (!file.is_open()) {
        cout << "Error: Unable to open audit log.\n";
        return;
    }
    auto now = chrono::system_clock::now();
    auto time = chrono::system_clock::to_time_t(now);
    file << put_time(localtime(&time), "%Y-%m-%d %H:%M:%S") << " | User: " <<
username
        << " | Action: " << action << "\n";
    file.close();
}

bool isValidInput(const string& input) {
    if (input.empty()) {
        cout << "Error: Input cannot be empty.\n";
        return false;
    }
    if (input.find(',') != string::npos) {
        cout << "Error: Input cannot contain commas.\n";
        return false;
    }
    return true;
}

```

```

bool userExists(const string& username, const vector<User>& users) {
    for (const auto& user : users) {
        if (user.username == username) {
            return true;
        }
    }
    return false;
}

```

```

vector<User> loadUsers() {
    vector<User> users;
    ifstream file("users.txt");
    if (!file.is_open()) {
        logAction("Failed to open users.txt for reading");
        return users;
    }
    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string username, password, role;
        getline(ss, username, ',');
        getline(ss, password, ',');
        getline(ss, role, ',');
        users.push_back({username, password, role});
    }
    file.close();
    logAction("Loaded users from file");
    return users;
}

```



```

void saveUsers(const vector<User>& users) {
    ofstream file("users.txt");
    if (!file.is_open()) {
        cout << "Error: Unable to open file for writing.\n";
        logAction("Failed to open users.txt for writing");
        return;
    }
    for (const auto& user : users) {
        file << user.username << "," << user.password << "," << user.role << "\n";
    }
    file.close();
    logAction("Saved users to file");
}

void registerUser(vector<User>& users) {
    string username, password, role;
    cout << "\n--- User Registration ---\n";
    cout << "Enter username: ";
    cin >> username;
    if (!isValidInput(username) || userExists(username, users)) {
        cout << "Error: Invalid or existing username.\n";
        logAction("Failed registration attempt: Invalid/existing username", username);
        return;
    }

    cout << "Enter password: ";
    cin >> password;
    if (!isValidInput(password)) {
        logAction("Failed registration attempt: Invalid password", username);
        return;
    }
}

```

```

    }

    cout << "Enter role (Admin/User/Guest): ";
    cin >> role;

    if (role != "Admin" && role != "User" && role != "Guest") {
        cout << "Error: Invalid role. Choose Admin, User, or Guest.\n";
        logAction("Failed registration attempt: Invalid role", username);
        return;
    }

    User newUser = {username, hashPassword(password), role};
    users.push_back(newUser);
    saveUsers(users);
    cout << "User registered successfully!\n";
    logAction("User registered", username);
}

User* login(vector<User>& users, Session& session) {
    string username, password;
    cout << "\n--- User Login ---\n";
    cout << "Enter username: ";
    cin >> username;
    cout << "Enter password: ";
    cin >> password;

    for (auto& user : users) {
        if (user.username == username && user.password == hashPassword(password)) {
            cout << "Login successful! Welcome, " << user.username << " (" << user.role <<
            ").\n";
            session.active = true;
            session.user = &user;
        }
    }
}

```

```

        session.lastActivity = chrono::system_clock::now();
        logAction("Successful login", username);
        return &user;
    }
}

cout << "Error: Invalid username or password.\n";
logAction("Failed login attempt", username);
return nullptr;
}

bool isSessionActive(Session& session) {
    if (!session.active) return false;

    auto now = chrono::system_clock::now();

    auto duration = chrono::duration_cast<chrono::seconds>(now -
session.lastActivity).count();

    if (duration > 300) { // 5 minutes
        cout << "Session timed out. Please log in again.\n";
        logAction("Session timeout", session.user ? session.user->username : "Unknown");
        session.active = false;
        session.user = nullptr;
        return false;
    }

    return true;
}

void viewAllUsers(const vector<User>& users, const string& adminUsername) {
    cout << "\n--- All Users ---\n";

    if (users.empty()) {
        cout << "No users found.\n";
        return;
    }
}

```

```

    cout << left << setw(20) << "Username" << setw(10) << "Role" << "\n";
    cout << string(30, '-') << "\n";
    for (const auto& user : users) {
        cout << left << setw(20) << user.username << setw(10) << user.role << "\n";
    }
    logAction("Viewed all users", adminUsername);
}

void viewProfile(const User& user) {
    cout << "\n--- User Profile ---\n";
    cout << "Username: " << user.username << "\n";
    cout << "Role: " << user.role << "\n";
    logAction("Viewed profile", user.username);
}

void updatePassword(vector<User>& users, User& currentUser) {
    string newPassword;
    cout << "\n--- Update Password ---\n";
    cout << "Enter new password: ";
    cin >> newPassword;
    if (!isValidInput(newPassword)) {
        logAction("Failed password update: Invalid password", currentUser.username);
        return;
    }

    for (auto& user : users) {
        if (user.username == currentUser.username) {
            user.password = hashPassword(newPassword);
            currentUser.password = user.password;
            break;
        }
    }
}

```

```

    }
}
saveUsers(users);
cout << "Password updated successfully!\n";
logAction("Password updated", currentUser.username);
}

void assignRole(vector<User>& users, const string& adminUsername) {
    string username, newRole;
    cout << "\n--- Assign Role ---\n";
    cout << "Enter username to assign role: ";
    cin >> username;
    cout << "Enter new role (Admin/User/Guest): ";
    cin >> newRole;

    if (newRole != "Admin" && newRole != "User" && newRole != "Guest") {
        cout << "Error: Invalid role.\n";
        logAction("Failed role assignment: Invalid role", adminUsername);
        return;
    }

    for (auto& user : users) {
        if (user.username == username) {
            user.role = newRole;
            saveUsers(users);
            cout << "Role updated successfully for " << username << "!\n";
            logAction("Assigned role " + newRole + " to " + username, adminUsername);
            return;
        }
    }
}

```

```

    cout << "Error: Username not found.\n";
    logAction("Failed role assignment: Username not found", adminUsername);
}

void guestMenu(Session& session) {
    cout << "\n--- Guest Menu ---\n";
    cout << "1. View Public Information\n";
    cout << "2. Exit\n";
    int choice;
    cout << "Enter choice: ";
    cin >> choice;
    if (!isSessionActive(session)) return;
    session.lastActivity = chrono::system_clock::now();
    if (choice == 1) {
        cout << "Public Information: This is a User Role Authentication System.\n";
        logAction("Viewed public info", session.user->username);
    }
}

```

```

void userMenu(vector<User>& users, Session& session) {
    while (isSessionActive(session)) {
        cout << "\n--- User Menu ---\n";
        cout << "1. View Profile\n";
        cout << "2. Update Password\n";
        cout << "3. Logout\n";
        int choice;
        cout << "Enter choice: ";
        cin >> choice;
        if (!isSessionActive(session)) return;
        session.lastActivity = chrono::system_clock::now();
    }
}

```

```

switch (choice) {
    case 1:
        viewProfile(*session.user);
        break;
    case 2:
        updatePassword(users, *session.user);
        break;
    case 3:
        cout << "Logging out...\n";
        logAction("Logged out", session.user->username);
        session.active = false;
        session.user = nullptr;
        return;
    default:
        cout << "Invalid choice.\n";
}
}
}

```

```

void adminMenu(vector<User>& users, Session& session) {
    while (isSessionActive(session)) {
        cout << "\n--- Admin Menu ---\n";
        cout << "1. View All Users\n";
        cout << "2. Assign Role\n";
        cout << "3. View Profile\n";
        cout << "4. Update Password\n";
        cout << "5. Logout\n";
        int choice;
        cout << "Enter choice: ";
        cin >> choice;
    }
}

```

```

if (!isSessionActive(session)) return;
session.lastActivity = chrono::system_clock::now();
switch (choice) {
    case 1:
        viewAllUsers(users, session.user->username);
        break;
    case 2:
        assignRole(users, session.user->username);
        break;
    case 3:
        viewProfile(*session.user);
        break;
    case 4:
        updatePassword(users, *session.user);
        break;
    case 5:
        cout << "Logging out...\n";
        logAction("Logged out", session.user->username);
        session.active = false;
        session.user = nullptr;
        return;
    default:
        cout << "Invalid choice.\n";
}
}
}

int main() {
    vector<User> users = loadUsers();
    Session session;

```



```

while (true) {

    cout << "\n--- User Role Authentication System ---\n";

    cout << "1. Register\n";

    cout << "2. Login\n";

    cout << "3. Exit\n";

    int choice;

    cout << "Enter choice: ";

    cin >> choice;


    switch (choice) {

        case 1:

            registerUser(users);

            break;

        case 2: {

            if (session.active) {

                cout << "Already logged in. Please log out first.\n";

                break;

            }

            User* loggedInUser = login(users, session);

            if (loggedInUser) {

                if (loggedInUser->role == "Admin") {

                    adminMenu(users, session);

                } else if (loggedInUser->role == "User") {

                    userMenu(users, session);

                } else if (loggedInUser->role == "Guest") {

                    guestMenu(session);

                }

            }

            break;

        }

    }

}

```

```
    case 3:
        cout << "Exiting system...\n";
        logAction("System exited");
        return 0;
    default:
        cout << "Invalid choice.\n";
    }
}
return 0;
}
```

TESTING

Testing is a crucial aspect of ensuring the reliability, security, and functionality of a user role authentication system. Objective of this is to ensure that passkey and biometric authentication methods work as expected, role-based access control is enforced correctly, and sensitive data such as passkeys and biometrics are handled securely. There are several methods like unit testing to validate individual components, functional testing to check core workflows like user registration and authentication, integration testing to verify that all system components work together seamlessly. Security testing was conducted to ensure no vulnerabilities, such as data leaks, exist, and usability testing was done to ensure that the authentication process is intuitive for users.

1. The testing process also prioritized real-world scenarios and edge cases, such as testing with invalid passkeys and biometric data.
2. Automated tests were used for repetitive checks, reducing the time needed for manual testing, while critical paths such as login and role-based access were tested manually to ensure accuracy. In addition, focus on testing for proper encryption and secure handling of sensitive data.

CONCLUSION

The User Role Authentication System successfully demonstrates the implementation of secure user authentication and role-based access control using C++. The project provides a structured mechanism for verifying user credentials and granting access according to predefined roles such as Admin, User, and Guest.

Through this project, important concepts of software security such as authentication, authorization, password hashing, session management, and data handling were explored and implemented effectively. The system ensures that only authorized users can access specific functionalities, thereby improving security and maintaining system integrity.

The project also highlights the practical use of Object-Oriented Programming concepts and file handling in C++. Features like user registration, login validation, role assignment, password updating, and audit logging make the system reliable and efficient. Testing confirmed that the application performs correctly under different scenarios and handles user access securely.

Overall, this project provides a strong foundation for understanding authentication systems and can be further enhanced in the future with advanced security features such as database integration, biometric authentication, OTP verification, cloud support, and stronger encryption techniques for real-world applications.

BIBLIOGRAPHY

The bibliography lists the books, websites, and other resources that were referred to while developing the To-Do List project in C++ and preparing its documentation.

- Balagurusamy, E. – Programming in ANSI C++, Tata McGraw-Hill Education.
- Herbert Schildt – C++: The Complete Reference, McGraw-Hill Education.
- Bjarne Stroustrup – The C++ Programming Language, Addison-Wesley.
- Object-Oriented Programming with C++ by Robert Lafore, SAMS Publishing. www.cplusplus.com

www.geeksforgeeks.org <https://www.programiz.com/cpp-programming>
<https://www.tutorialspoint.com/cplusplus>